

Aula 05

CRUD Completo

Período: 6º semestre

Carga horária: 60h

Código Classroom: cxuqp2it

Professor: Braully Rocha da Silva

Objetivos

Completar o back-end com os métodos PUT (atualizar) e DELETE (excluir) Criar formulários controlados em React para cadastrar e editar Usuario Integrar front-end e back-end em um CRUD completo (Cadastro · Listagem · Alteração · Exclusão)

Revisão da Aula Anterior

Aprendemos os fundamentos do React: componentes, props, state e hooks (`useState` , `useEffect`)

Configuramos o Axios (`api.ts`) e o CORS na classe `Api`

Criamos a tela de listagem `UsuarioList` consumindo `GET /api/usuarios` No back-end (Aula 03), já existem `GET /api/usuarios` e `POST /api/usuarios` com persistência no H2

Hoje: fechar o ciclo do CRUD com **atualizar** (`PUT`) e **excluir** (`DELETE`), além de formulários no React.

Conceitos Fundamentais

O que é CRUD?

CRUD são as quatro operações básicas de persistência de dados:

Create — criar (inserir um novo registro)

Read — ler (listar/buscar registros)

Update — atualizar (alterar um registro existente)

Delete — excluir (remover um registro)

Cada operação é mapeada para um **método HTTP** e para um método do `JpaRepository` .

CRUD × HTTP × Repository

Operação	Método HTTP	Endpoint	Repository
Create	POST	/api/usuarios	save()
Read (todos)	GET	/api/usuarios	findAll()
Read (um)	GET	/api/usuarios/{id}	findById()
Update	PUT	/api/usuarios/{id}	save()
Delete	DELETE	/api/usuarios/{id}	deleteById()

No PUT , o save() é usado sobre um objeto já existente (recuperado por findById), então ele **atualiza** em vez de inserir.

Formulários Controlados (React)

Um **input controlado** tem o `value` ligado ao `state` e um `onChange` que atualiza esse `state` a cada digitação

O React é a "fonte da verdade": o valor exibido vem sempre do `state`

```
const [nome, setNome] = useState("");
```

```
<input  
  value={nome}  
  onChange={(evento) => setNome(evento.target.value)}  
>
```

No envio (`onSubmit`), os valores do `state` são enviados para a API via POST (novo) ou PUT (edição)

Arquitetura / Fluxo

React — UsuarioForm (formulário controlado)

POST /api/usuarios (cadastrar)
PUT /api/usuarios/{id} (editar)
DELETE /api/usuarios/{id} (excluir)

▼
Api (@RestController)

▼
UsuarioRepository (Spring Data JPA)

▼
Banco H2

Resposta JSON —► carregarUsuarios() —► re-render da lista

Após cada operação de escrita (criar, editar, excluir), recarregamos a lista

chamando novamente GET /ai/usuarios .

Programação Web II

Demonstração — Api completo (Api.java)

```
@RestController
@RequestMapping("/api")
@CrossOrigin(origins = "http://localhost:5173")
public class Api {

    @Autowired
    private UsuarioRepository usuarioRepository;

    // CREATE
    @PostMapping("/usuarios")
    public Usuario createUsuario(@RequestBody Usuario usuario) {
        return usuarioRepository.save(usuario);
    }

    // READ — todos
    @GetMapping("/usuarios")
    public List<Usuario> getAllUsuarios() {
        return usuarioRepository.findAll();
    }
}
```

```
// READ — por id
@GetMapping("/usuarios/{id}")
public Usuario getUsuarioById(@PathVariable Long id) {
    return usuarioRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Usuário não encontrado"));
}

// UPDATE
@PutMapping("/usuarios/{id}")
public Usuario updateUsuario(@PathVariable Long id, @RequestBody Usuario usuarioAtualizado) { Usuario usuario =
    usuarioRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Usuário não encontrado"));

    usuario.setNome(usuarioAtualizado.getNome());
    usuario.setUsername(usuarioAtualizado.getUsername());
    usuario.setEmail(usuarioAtualizado.getEmail());

    return usuarioRepository.save(usuario);
}

// DELETE
@DeleteMapping("/usuarios/{id}")
public void deleteUsuario(@PathVariable Long id) {
    usuarioRepository.deleteById(id);
}
}
```

A senha não é alterada no PUT porque ela está com `@JsonIgnore` e não 8

Programação Web II

Demonstração — `UsuarioForm.tsx` (criar/editar)

```
// src/components/UsuarioForm.tsx
import { FormEvent, useState } from "react";
import api from "../services/api";
import type { Usuario } from "../types/Usuario";

interface UsuarioFormProps {
  onUsuarioSalvo: () => void;
  usuarioEditando?: Usuario | null;
}

function UsuarioForm({ onUsuarioSalvo, usuarioEditando }: UsuarioFormProps) {
  const [nome, setNome] = useState(usuarioEditando?.nome ?? "");
  const [username, setUsername] = useState(usuarioEditando?.username ?? "");
  const [email, setEmail] = useState(usuarioEditando?.email ?? "");

  async function handleSubmit(event: FormEvent) {
    event.preventDefault();

    const dados = { nome, username, email };

    if (usuarioEditando) {
      await api.put(`/usuarios/${usuarioEditando.id}`, dados);
    } else {
```

```

await api.post("/usuarios", dados);
}

onUsuarioSalvo();

}

```

ProgramaçãoWebII

```

return(
  <formonSubmit={handleSubmit}>
    <input
      value={nome}
      onChange={(e)=>setNome(e.target.value)}
      placeholder="Nome"
    />
    <input
      value={username}
      onChange={(e)=>setUsername(e.target.value)}
      placeholder="Username"
    />
    <input
      value={email}
      onChange={(e)=>setEmail(e.target.value)}
      placeholder="E-mail"
    />
    <buttontype="submit">
      {usuarioEditando?"Salvaralterações" : "Cadastrar"}
    </button>
  </form>
);

```

```
}
```

```
export default UsuarioForm;
```

10

Programação Web II

Demonstração — UsuarioList.tsx (com editar e excluir)

```
// src/components/UsuarioList.tsx
import { useEffect, useState } from "react";
import api from "../services/api";
import type { Usuario } from "../types/Usuario";
import UsuarioItem from "../UsuarioItem";
import UsuarioForm from "../UsuarioForm";

function UsuarioList() {
  const [usuarios, setUsuarios] = useState<Usuario[]>([]);
  const [editando, setEditando] = useState<Usuario | null>(null);

  function carregarUsuarios() {
    api.get<Usuario[]>("/usuarios").then((resposta) => {
      setUsuarios(resposta.data);
    });
  }

  useEffect(() => {
    carregarUsuarios();
  }, []);
}
```

```
async function excluir(id: number) {  
  await api.delete(`/usuarios/${id}`);  
  carregarUsuarios();  
}
```

Programação Web II

```
return (  
  <div>  
    <UsuarioForm  
      key={editando?.id ?? "novo"}  
      usuarioEditando={editando}  
      onUsuarioSalvo={() => {  
        carregarUsuarios();  
        setEditando(null);  
      }}  
    />  
  
    <ul>  
      {usuarios.map((usuario) => (  
        <li key={usuario.id}>  
          <UsuarioItem usuario={usuario} />  
          <button onClick={() => setEditando(usuario)}>Editar</button>  
          <button onClick={() => excluir(usuario.id)}>Excluir</button>  
        </li>  
      ))}  
    </ul>  
  </div>
```

```
    );  
  }  
  
  export default UsuarioList;
```

A propriedade key no UsuarioForm força o React a **recriar o formulário** 12

Programação Web II

Demonstração — App.tsx

```
import UsuarioList from "../components/UsuarioList";  
  
function App() {  
  return (  
    <div>  
      <h1>Usuários</h1>  
      <UsuarioList />  
    </div>  
  );  
}
```

```
export default App;
```

Atividade Prática

Objetivo

Completar o CRUD de Usuario , permitindo cadastrar, listar, editar e excluir usuários pela interface React.

Passos

1. Adicionar `@PutMapping("/usuarios/{id}")` e `@DeleteMapping("/usuarios/{id}")` na classe `Api`, usando `@PathVariable Long id` (nunca `@RequestBody` para o id , pois GET / DELETE não têm corpo)
2. Criar o componente `UsuarioForm.tsx` com formulário controlado (nome, username, email)
3. Atualizar `UsuarioList.tsx` para incluir os botões **Editar** e **Excluir** e recarregar a lista após cada operação
4. Testar

o fluxo completo no navegador: cadastrar → editar → excluir (com o back-end rodando)

5. Verificar no H2 Console se os dados estão sendo persistidos corretamente

Exercícios

1. Completar o CRUD da Permissao : adicionar PUT e DELETE no PermissaoController e criar PermissaoForm + botões de editar/excluir no React
2. Completar o CRUD da entidade de seu interesse (Produto , Pedido , Cliente , etc.), replicando o mesmo fluxo
3. Ao final, comitar e subir as alterações no repositório do GitHub

Referências

Spring Boot Documentation

Spring Data JPA Documentation

React Documentation — <https://react.dev/learn>

Axios Documentation — <https://axios-http.com/docs/intro>

Material complementar da disciplina

Próxima Aula

Conteúdo: Arquitetura em Camadas — Service · Repository · Controller no Spring e organização de pastas no React

Preparação: deixar o CRUD completo de Usuario funcionando, com back-end e front-end integrados

